

Lab 9 Sets Ops, Where Clause Subqueries

United International College
COMP3013 DBMS

Outline

- Set Operations
 - Union
 - Intersection
 - Set difference
- Subquery
 - IN subquery
 - NOT IN subquery

Set Operations

Tables are treated as sets in relational databases.

So, set operations are defined in SQL.

Let A and B be two sets.

- The **union** of A and B is the set $A \cup B = \{t | t \in A \vee t \in B\}$.
- The **intersection** of A and B is the set $A \cap B = \{t | t \in A \wedge t \in B\}$.
- The **set difference** of A and B is the set $A - B = \{t | t \in A \wedge t \notin B\}$.

- For example, $A = \{x, 1\}$, $B = \{1, y\}$
 - $A \cup B = \{x, 1, y\}$
 - $A \cap B = \{1\}$
 - $A - B = \{x\}$.
- For SQL, each element in the set is a tuple (table row).

Set Operations

Let $r = (A, B)$ and $s = (A, B)$ be two relations in database.

r

A	B
α	1
α	2
β	1

s

A	B
α	2
β	3

Note:

Duplicated rows are removed in this example (because sets do not have duplications).

But, MySQL keeps duplications.

$r \cup s$

A	B
α	1
α	2
β	1
β	3

$r \cap s$

A	B
α	2

$r - s$

A	B
α	1
β	1

Union

- Find the title of films which is played by “Bob” or “Zero” (first names of actors).
 1. Select the films played by Bob
 2. Select the films played by Zero
 3. Union the two tables

```
(SELECT title, first_name  
FROM actor JOIN film_actor USING (actor_id) JOIN film USING (film_id) WHERE  
first_name='Bob')
```

UNION

```
(SELECT title, first_name  
FROM actor JOIN film_actor USING (actor_id) JOIN film USING (film_id) WHERE  
first_name='Zero')
```

Union

Please be careful when UNION is used.

- The two tables of the union must be **compatible**: corresponding columns must be of the same **type**.
- To check the type of columns, click “structure” tag for each table.
- Types are formally introduced in the following lab.
- MySQL is very robust. It allows union between different types.
- For example,

```
(SELECT actor_id FROM actor) UNION (SELECT first_name FROM actor)
```
- However, this is NOT the reason for a union without type checking.

Intersection

- For table intersection use the keyword **INTERSECT**.
- Find the id of the English films which are played by Tim Hackman.

```
(SELECT film_id
FROM film_actor JOIN actor USING(actor_id)
WHERE first_name='Tim' AND last_name='Hackman')
INTERSECT
(SELECT film_id
FROM film JOIN language USING(language_id)
WHERE name='English')
```

- Previous versions of MySQL doesn't support **INTERSECT**. In that case, we can use **IN** subquery instead. We will learn that later in this lab.

Set Difference

- For set difference use the keyword **EXCEPT**.
- Find the id of the films which are played by Tim Hackman but not in English.

```
(SELECT film_id
FROM film_actor JOIN actor USING(actor_id)
WHERE first_name='Tim' AND last_name='Hackman')
EXCEPT
(SELECT film_id
FROM film JOIN language USING(language_id)
WHERE name='English')
```

- Previous versions of MySQL doesn't support INTERSECT. In that case, we can use NOT IN subquery instead. We will learn that later in this lab.

Set Operations

- Sometimes set operations can be implemented without using the previous keywords.
- Let $A = \{t|P_A(t)\}$ and $B = \{t|P_B(t)\}$, then for example
$$A \cup B = \{t|P_A(t) \vee P_B(t)\}$$
- So, we change the predicate in the WHERE clause
$$\text{SELECT ... FROM fromClause WHERE } P_A \text{ OR } P_B$$
- Provided the from clause is the same for A and B.
- Similarly for intersection and set difference.

Subquery

- Subqueries can also be used to test the **membership** of one tuple (element) in one table (set).
- We can use the **IN** subquery for intersection and **NOT IN** subquery for set difference.
- The intersection $A \cap B$ is defined as $\forall x, x \in A \wedge x \in B$.
- Which is equivalent to $\forall x \in A, x \in B$ (by changing the domain of x).

IN Subquery for INTERSECTION

- Consider the SQL statement for intersect ($\forall x, x \in A \wedge x \in B$)

```
(SELECT columns FROM tables1 WHERE P1)
INTERSECT
(SELECT columns FROM tables2 WHERE P2)
```

- The corresponding **IN** subquery is ($\forall x \in A, x \in B$)

```
SELECT columns FROM tables1
WHERE P1 AND columns IN (
    SELECT columns FROM tables2 WHERE P2
)
```

Intersection Example

- Find the id of the English films which are played by Tim Hackman.
- We can use INTERSECT in the latest version of MySQL.

```
(SELECT film_id
FROM film JOIN language USING(language_id)
WHERE name='English')
INTERSECT
(SELECT film_id
FROM film_actor JOIN actor USING(actor_id)
WHERE first_name='Tim' AND last_name='Hackman')
```

- As an alternative to INTERSET, we can make the **second SELECT** into a subquery.

Intersection Example

- Using IN subquery instead of INTERSECTION

```
SELECT film_id
FROM film JOIN language USING(language_id)
WHERE name = 'English' AND
      film_id IN (
        SELECT film_id
        FROM film_actor JOIN actor USING(actor_id)
        WHERE first_name='Tim' AND last_name='Hackman'
      )
```

- The **subquery** constructs a temporary table of ids of the films played by Tim Hackman.
- The keyword **IN** tests whether the value of film id for each tuple is in the temporary table.

SOME

- The previous example can also be implemented by using SOME.
- If an element is in a set, then the element **is equal to some** element in the set.

```
SELECT film_id
FROM film JOIN language USING(language_id)
WHERE name = 'English' AND
      film_id = SOME(
        SELECT film_id
        FROM film_actor JOIN actor USING(actor_id)
        WHERE first_name='Tim' AND last_name='Hackman'
      )
```

NOT IN Subquery for EXCEPT (Set Difference)

- Same for set difference, $A - B$ is defined as $\forall x \in A, x \notin B$.
- Consider the SQL statement for Set Difference

```
(SELECT columns FROM tables1 WHERE P1) EXCEPT  
(SELECT columns FROM tables2 WHERE P2)
```

- Alternatively we can use the **NOT IN** subquery.

```
SELECT columns FROM tables1  
WHERE P1 AND columns NOT IN (  
    SELECT columns FROM tables2 WHERE P2  
)
```

Set Difference Example

- Find the id of the films which are played by Tim Hackman that are not in English.
- In the latest version of MySQL, we can use EXCEPT.

```
(SELECT film_id
FROM film_actor JOIN actor USING(actor_id)
WHERE first_name='Tim' AND last_name='Hackman')
EXCEPT
(SELECT film_id
FROM film JOIN language USING(language_id)
WHERE name='English')
```


Set Difference Example

- We can use NOT IN subquery instead of EXCEPT

```
SELECT film_id
FROM film_actor JOIN actor USING(actor_id)
WHERE first_name='Tim' AND last_name='Hackman' AND
      film_id NOT IN (
        SELECT film_id
        FROM film JOIN language USING(language_id)
        WHERE name='English')
```

- The **subquery** constructs a temporary table of ids of films in English.
- The keyword **NOT IN** tests whether the value of film id for each tuple is not in the temporary table, i.e. not in English.

Exercises

Write MySQL queries for the following:

1. In the Example (pg. 5) of Set Operation for UNION, rewrite the query with set operation without the keyword UNION.
2. In the Example for Intersect, “Find the id of the English films which are played by Tim Hackman”, the query result only show the film_id. Rewrite the query to display the language as well in the result.
3. In the Example for Set Difference, “Find the id of the films which are played by Tim Hackman that are not in English”, the result seems to be empty. Verify this by writing a query to display the film_id and language of all films played by Tim Hackman.
4. Find the id of the customers who live in Australia that have not rented any film.

Save your queries in a txt file. Rename it as “COMP3013 Lab9 ####.txt”, where “####” is your student ID. And submit it on iSpace. The DDL is 24 hours after the lab.

End of Lab 9